

Machine Learning Classification

Notes

1. What is Classification?

Classification is a **Supervised Machine Learning** technique where the model predicts a **category or class label**.

Examples

Problem	Output
Email Spam Detection	Spam / Not Spam
Disease Prediction	Diseased / Healthy
Sentiment Analysis	Positive / Negative
Titanic Survival	Survived / Not Survived

Types of Classification

Binary Classification

Only 2 classes

Examples:

- Yes / No
- Spam / Not Spam
- Survived / Not Survived

Multi-Class Classification

More than 2 classes

Examples:

- Cat / Dog / Horse
 - Grade A / B / C
-

Dataset Setup

```
import pandas as pd

df = pd.read_csv("data.csv")

X = df.drop("target", axis=1)
y = df["target"]
```

Train-Test Split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

2. Logistic Regression

What is Logistic Regression?

Used for Binary Classification.

Predicts probability between 0 and 1.

Example:

0.85	Survived
0.20	Not Survived

Sigmoid Function

Converts any value into probability.

[$\sigma(z)=\frac{1}{1+e^{-z}}$]

Output Range:

0 to 1

Code

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()

model.fit(X_train, y_train)

pred = model.predict(X_test)
```

Advantages

Fast

Easy to understand

Works well for binary problems

Disadvantages

Struggles with complex nonlinear data

3. K-Nearest Neighbors (KNN)

Concept

"Similar things are close together."

When a new point comes:

1. Find nearest K neighbors
 2. Count classes
 3. Majority wins
-

Example

K = 5

Dog
Dog
Dog
Cat
Cat

Prediction = Dog

Code

```
from sklearn.neighbors import KNeighborsClassifier  
  
model = KNeighborsClassifier(n_neighbors=5)
```

```
model.fit(X_train, y_train)

pred = model.predict(X_test)
```

Advantages

Simple

No training required

Disadvantages

Slow for large datasets

Sensitive to feature scaling

Feature Scaling

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

4. Decision Tree

Concept

Makes decisions using questions.

Example:

Age > 18 ?

Yes

|

Income > 50k ?

Yes

|

Buy Product

Important Terms

Root Node

Starting point of tree.

Internal Node

Decision point.

Leaf Node

Final output.

Code

```
from sklearn.tree import DecisionTreeClassifier
```

```
model = DecisionTreeClassifier()
```

```
model.fit(X_train, y_train)
```

```
pred = model.predict(X_test)
```

Visualize Tree

```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

plot_tree(model)

plt.show()
```

Advantages

Easy to understand

Handles nonlinear relationships

Disadvantages

Overfitting

5. Naive Bayes

Concept

Based on Bayes Theorem.

Assumes all features are independent.

Bayes Formula

[$P(A | B) = \frac{P(B | A)P(A)}{P(B)}$]

Applications

- Spam Detection
 - Text Classification
 - Sentiment Analysis
-

Code

```
from sklearn.naive_bayes import GaussianNB

model = GaussianNB()

model.fit(X_train, y_train)

pred = model.predict(X_test)
```

Advantages

Very fast

Great for text data

Disadvantages

Independence assumption rarely true

6. Support Vector Machine (SVM)

Concept

Find the best line (hyperplane) that separates classes.

Example

Class A | Margin | Hyperplane | Margin | Class B

Important Terms

Hyperplane

Decision boundary.

Margin

Distance from boundary.

Support Vectors

Closest points to boundary.

Code

```
from sklearn.svm import SVC

model = SVC(kernel="linear")

model.fit(X_train, y_train)

pred = model.predict(X_test)
```

RBF Kernel

Used for nonlinear data.

```
model = SVC(kernel="rbf")
```

Advantages

Works well on small datasets

Powerful classifier

Disadvantages

Slow on huge datasets

Evaluation Metrics

7. Accuracy

Measures overall correctness.

Formula:

[$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$]

Code

```
from sklearn.metrics import accuracy_score

accuracy_score(y_test, pred)
```

8. Precision

Among predicted positives, how many are actually positive?

Formula:

[$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$]

Code

```
from sklearn.metrics import precision_score  
  
precision_score(y_test, pred)
```

Example Use Case

Spam Detection

Avoid marking normal emails as spam.

9. Recall

Among actual positives, how many did we catch?

Formula:

[$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$]

Code

```
from sklearn.metrics import recall_score  
  
recall_score(y_test, pred)
```

Example Use Case

Cancer Detection

Missing a patient is dangerous.

10. F1 Score

Balance between Precision and Recall.

Formula:

$$[F1 = \frac{2(P \times R)}{P + R}]$$

Code

```
from sklearn.metrics import f1_score  
  
f1_score(y_test, pred)
```

11. Confusion Matrix

Most Important Interview Topic

Structure

	Predicted Yes	Predicted No
Actual Yes	TP	FN
Actual No	FP	TN

Meanings

TP

Actual Positive and Predicted Positive

TN

Actual Negative and Predicted Negative

FP

Actual Negative but Predicted Positive

FN

Actual Positive but Predicted Negative

Code

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, pred)

print(cm)
```

Visualization

```
from sklearn.metrics import ConfusionMatrixDisplay

ConfusionMatrixDisplay.from_predictions(
    y_test,
    pred
)
```

Classification Report

Provides all metrics together.

```
from sklearn.metrics import classification_report

print(
    classification_report(
        y_test,
        pred
    )
)
```

Output:

```
Precision
Recall
F1-score
Support
```

Titanic Survival Classification Project

Objective

Predict whether a passenger survived.

Target Column:

```
Survived
```

Features

```
Age
Sex
Fare
Pclass
```

```
SibSp  
Parch
```

Step 1: Import Libraries

```
import pandas as pd
```

Step 2: Load Dataset

```
df = pd.read_csv("Titanic.csv")
```

Step 3: Select Features

```
features = [  
    "Pclass",  
    "Age",  
    "Fare"  
]  
  
X = df[features]  
  
y = df["Survived"]
```

Step 4: Handle Missing Values

```
X.fillna(X.mean(), inplace=True)
```

Step 5: Train-Test Split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

Step 6: Train Logistic Regression

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()

model.fit(X_train, y_train)
```

Step 7: Prediction

```
pred = model.predict(X_test)
```

Step 8: Evaluation

```
from sklearn.metrics import accuracy_score

accuracy_score(y_test, pred)
```

Interview Revision (1-Minute)

Logistic Regression

- Binary Classification
- Uses Sigmoid Function

KNN

- Majority Voting
- Needs Scaling

Decision Tree

- Tree Structure
- Can Overfit

Naive Bayes

- Bayes Theorem
- Great for Text Data

SVM

- Best Margin
- Uses Hyperplane

Metrics

- Accuracy Overall Correctness
- Precision FP Important
- Recall FN Important
- F1 Balance
- Confusion Matrix TP, TN, FP, FN